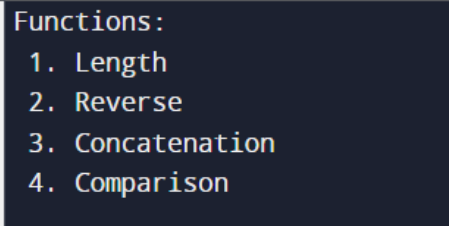


Упражнение 2 - Втора част

1. зад. Да се създадат 4 функции – описани на фиг.1



```
Functions:
1. Length
2. Reverse
3. Concatenation
4. Comparison
```

фиг.1

Първата при въвеждане на низ от клавиатурата да извежда дължината на му.

Втората при въвеждане на низ от клавиатурата да извежда низа в обратна посока.

Третата при въвеждане на два низа от клавиатурата да ги слепва.

Четвъртата при въвеждане на два низ от клавиатурата да ги сравнява и да извежда подходящи съобщения.

Забележка: виж. фиг.3

Кодиране:

```
def strlen(str):
```

```
    counter = 0
```

```
    while str[counter:]:
```

```
        counter += 1
```

```
    return counter
```

```
def strrev(str):
```

```
    rstr=""
```

```
    l=strlen(str)
```

```
    while l>0:
```

```
        rstr = rstr + str[l-1]
```

```
        l=l-1
```

```
    return rstr
```

```
def strcat(st1,st2):
```

```
    return(st1+st2)
```

```
def strcmp(st1,st2):
```

```
    if(st1==st2):
```

```
        print(st1 + " and " + st2 + " are same")
```

```
    elif (st1>st2):
```

```
        print(st1 + " comes after " + st2 +" in the Dictionary")
```

```
    else:
```

```
        print(st1 + " comes before " + st2 +" in the Dictionary")
```

```
print("Functions:\n 1. Length \n 2. Reverse \n 3. Concatenation\n"+
```

```
" 4. Comparison\n")
```

```
n=int(input("Enter your Choice: "))
```

```
if(n==1):
```

```
    str = input("Enter a String: ")
```

```
    print("Length of the string is:", strlen(str))
```

```
elif (n==2):
```

```
    str = input("Enter a String: ")
```

```
    print("Reversed String is:", strrev(str))
```

```
elif (n==3):
```

```
    str1 = input("Enter the first String: ")
```

```

str2 = input("Enter the second String: ")
print("Concatenated string is:", strcat(str1,str2))
elif (n==4):
    str1 = input("Enter the first String: ")
    str2 = input("Enter the second String: ")
    strcmp(str1,str2)
else:
    print("Invalid Choice")

```

Резултат:

```

main.py
1 - def strlen(str):
2     counter = 0
3     while str[counter:]:
4         counter += 1
5     return counter
6
7 - def strrev(str):
8     rstr=""
9     l=len(str)
10    while l>0:
11        rstr = rstr + str[l-1]
12        l=l-1
13    return rstr
14
15 - def strcat(st1,st2):
16     return(st1+st2)
17
18 - def strcmp(st1,st2):
19     if(st1==st2):
20         print(st1 + " and " + st2 + " are same")
21     elif (st1>st2):
22         print(st1 + " comes after " + st2 + " in the Dictionary")
23     else:
24         print(st1 + " comes before " + st2 + " in the Dictionary")
25 print("Functions:\n 1. Length \n 2. Reverse \n 3. Concatenation\n"
26       " 4. Comparison\n")
27 n=int(input("Enter your Choice: "))
28 if(n==1):
29     str = input("Enter a String: ")
30     print("Length of the string is:", strlen(str))
31 - elif (n==2):
32     str = input("Enter a String: ")
33     print("Reversed String is:", strrev(str))
34 - elif (n==3):
35     str1 = input("Enter the first String: ")
36     str2 = input("Enter the second String: ")
37     print("Concatenated string is:", strcat(str1,str2))
38 - elif (n==4):
39     str1 = input("Enter the first String: ")
40     str2 = input("Enter the second String: ")
41     strcmp(str1,str2)
42 else:
43     print("Invalid Choice")

```

фиг.2

Избор на функция и въвеждане на входни данни:

Functions: 1. Length 2. Reverse 3. Concatenation 4. Comparison Enter your Choice: 1 Enter a String: qwerty Length of the string is: 6	Functions: 1. Length 2. Reverse 3. Concatenation 4. Comparison Enter your Choice: 2 Enter a String: qwerty Reversed String is: yterwq	Functions: 1. Length 2. Reverse 3. Concatenation 4. Comparison Enter your Choice: 3 Enter the first String: qwerty Enter the second String: qwerty Concatenated string is: qwertyqwerty	Functions: 1. Length 2. Reverse 3. Concatenation 4. Comparison Enter your Choice: 4 Enter the first String: qwerty Enter the second String: qwerty qwerty and qwerty are same
---	---	--	--

фиг.3

2. зад. Като използвате описаните в Таблица 1 функции, създайте приложение с меню и при избор на функция и въведени входни данни изведете резултата от изпълнението и.

Таблица 1

Функции	Описание
>>>max(3,4) 4	# връща най-големия елемент
>>>len("hello") 5	# връща дължината на обект
>>>range(2,8,1) [2, 3, 4, 5, 6, 7]	# връща диапазон от дадени стойности

>>>round(7.8) 8.0	# връща закръглено цяло число от даденото число
>>>chr(102) f	# връща символ (низ) от цяло число
>>>float(5) 5.0	# връща плаващо число от низ или цяло число
>>>int(5.0) 5	# връща цяло число от низ или float
>>>pow(3,5) 243	# връща число повдигнато на степен
>>>type(5.6) <type 'float'>	# връща тип данни на обекта, към който принадлежи
>>>t=tuple([4,6.0,7]) (4, 6.0, 7)	# за да създадете набор от елементи от списъка
>>>print("good morning") Good morning	# показва дадения обект

3. зад. Въведете следният код и опишете, какво е предназначението му.

```
# Python Program to
# show range() basics
# printing a number
for i in range(10):
    print(i, end=" ")
print()
# using range for iteration
l = [10, 20, 30, 40]
for i in range(len(l)):
    print(l[i], end=" ")
print()
# performing sum of natural
# number
sum = 0
for i in range(1, 11):
    sum = sum + i
print("Sum of first 10 natural number :", sum)
```

```
1 # Python Program to
2 # show range() basics
3 # printing a number
4 for i in range(10):
5     print(i, end=" ")
6 print()
7
8 # using range for iteration
9 l = [10, 20, 30, 40]
10 for i in range(len(l)):
11     print(l[i], end=" ")
12 print()
13
14 # performing sum of natural
15 # number
16 sum = 0
17 for i in range(1, 11):
18     sum = sum + i
19 print("Sum of first 10 natural number :", sum)
```

фиг.4

```
0 1 2 3 4 5 6 7 8 9
10 20 30 40
Sum of first 10 natural number : 55
> |
```

фиг.5

4. зад. Да се създадат различни видове кортежи: празен, кортеж с цели числа, кортеж със смесен тип данни, вложен кортеж

```
my_tuple = ()
print(my_tuple)
```

```
# Tuple having integers
my_tuple = (1, 2, 3)
print(my_tuple)
```

```
# tuple with mixed datatypes
my_tuple = (1, "Hello", 3.4)
print(my_tuple)
```

```
# nested tuple
my_tuple = ("mouse", [8, 4, 6], (1, 2, 3))
print(my_tuple)
```

Резултат:

```
1 # Different types of tuples
2 # Empty tuple
3 my_tuple = ()
4 print(my_tuple)
5
6 # Tuple having integers
7 my_tuple = (1, 2, 3)
8 print(my_tuple)
9
10 # tuple with mixed datatypes
11 my_tuple = (1, "Hello", 3.4)
12 print(my_tuple)
13
14 # nested tuple
15 my_tuple = ("mouse", [8, 4, 6], (1, 2, 3))
16 print(my_tuple)
```

```
()
(1, 2, 3)
(1, 'Hello', 3.4)
('mouse', [8, 4, 6], (1, 2, 3))
> |
```

фиг.6

5. зад. Като използвате следна функция, допълнете с действие: изваждане, умножение, деление, целочислено деление, степенуване, коренуване.

```
def add(a,b):
    c = a + b
    return(c)
c=add(a,b)
print("Result:", add(2,2))
```

main.py	Run	Shell
<pre>1 def add(a,b): 2 c = a + b 3 return(c) 4 c=add(2,2) 5 print("Result:", c)</pre>		<pre>Result: 4 > </pre>

фиг.7

6.зад. Като използвате модул random и метода string.ascii_lowercase, създайте приложение чрез което генерира случаен низ с фиксирана дължина състоящ се само от малки букви

Решение:

```
import random
import string
def randomString(stringLength=10):
    """Generate a random string of fixed length """
    letters = string.ascii_lowercase # с използване на малки букви
    return ''.join(random.choice(letters)
for i in range(stringLength))
print ("Random String is ", randomString() )
print ("Random String is ", randomString(10) )
print ("Random String is ", randomString(10) )
```

Резултат:

Random String is efnmyehray

Random String is hs

Random String is hgpshamrh

Random String is irtznkfpwzu

7.зад. Да се генерира произволен низ само от конкретни букви:

Решение:

```
import random
import string
# Генерирайте произволен низ от конкретни букви
def randString(length=5):
#поставете в your_letters следното
    your_letters='abcdefghi'
    return ''.join((random.choice(your_letters)
for i in range(length)))
print ("Random String with specific letters ", randString() )
print ("Random String with specific letters ", randString(5) )
```

Резултат:

Random String with specific letters agbfh

Random String with specific letters deifd

8.зад. Генериране на произволен низ с букви и цифри в Python

```
import random
import string
def randomStringDigits(stringLength=6):
    """Generate a random string of letters and digits """
    lettersAndDigits = string.ascii_letters + string.digits
    return ''.join(random.choice(lettersAndDigits)
for i in range(stringLength))
print ("Generating a Random String including letters and digits")
print ("First Random String is ", randomStringDigits(8))
print ("Second Random String is ", randomStringDigits(8))
print ("Third Random String is ", randomStringDigits(8))
```

Резултат:

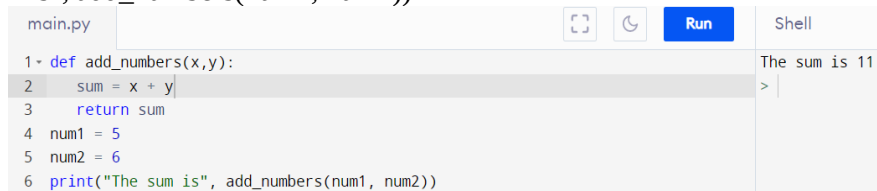
Generating a Random String including letters and digits

First Random String is mnnCYYbe

Second Random String is AWoWMP13
Third Random String is tjMydzZq

9.зад. Разгледайте примера и съпоставете с този от задача 5.

```
# Program to illustrate
# the use of user-defined functions
def add_numbers(x,y):
    sum = x + y
    return sum
num1 = 5
num2 = 6
print("The sum is", add_numbers(num1, num2))
```

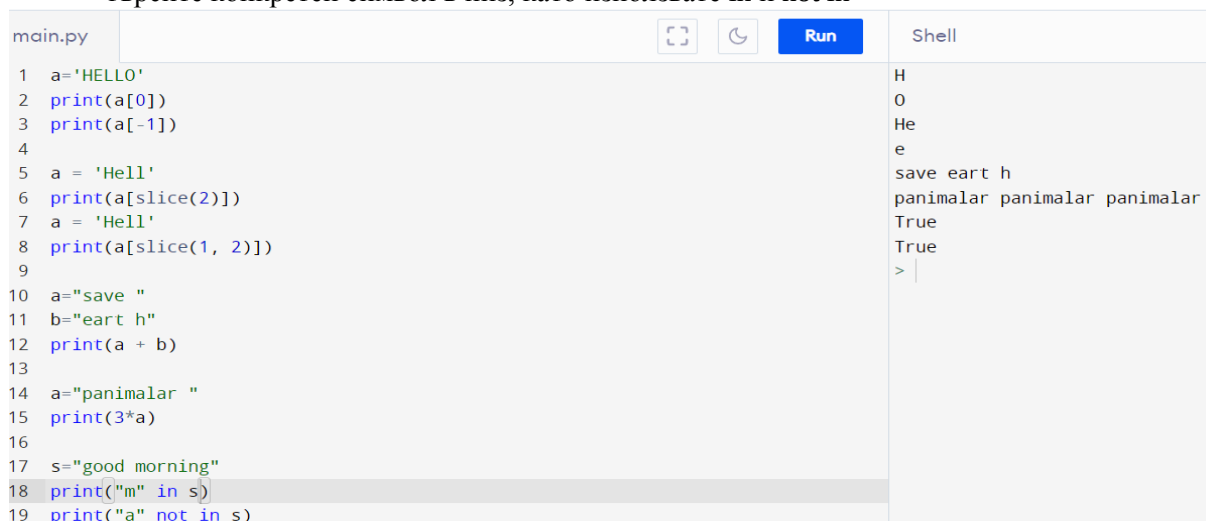


```
main.py  Run Shell
1 def add_numbers(x,y):
2     sum = x + y
3     return sum
4 num1 = 5
5 num2 = 6
6 print("The sum is", add_numbers(num1, num2))
The sum is 11
> |
```

фиг.8

10. зад. Като използвате фиг.9 създайте приложение, при което въвеждате низ от клавиатурата и определяте:

- броя на символите,
- връщате индекса на първият и последният символ,
- връщате Акси кода на първия и последния символ,
- удвоявате символа,
- търсите конкретен символ в низ, като използвате in и not in



```
main.py  Run Shell
1 a='HELLO'
2 print(a[0])
3 print(a[-1])
4
5 a = 'Hell'
6 print(a[slice(2)])
7 a = 'Hell'
8 print(a[slice(1, 2)])
9
10 a="save "
11 b="eart h"
12 print(a + b)
13
14 a="panimalar "
15 print(3*a)
16
17 s="good morning"
18 print("m" in s)
19 print("a" not in s)
H
O
He
e
save eart h
panimalar panimalar panimalar
True
True
> |
```

фиг.9

11.зад. Опишете резултата от форматирането:

```
print ('{:_<10}'.format('test'))
print ('{:d}'.format(42))
print ( '{:10.5}'.format('xylophone'))
print ( '{:>10}'.format('test'))
print ( '{:10}'.format('test'))
print ('{:^10}'.format('test'))
print ( '{:^6}'.format('zip'))
print ('{:f}'.format(3.141592653589793))
print ('{:4d}'.format(42))
print ('{:06.2f}'.format(3.141592653589793))
```

```
print('{:+d}'.format(42))
print('{: d}'.format((- 23)))
print('% d' % (42,))
data = {'first': 'Hodor', 'last': 'Hodor!'}
print ('{first} {last}'.format(**data))
print ('{first} {last}'.format(first='Hodor', last='Hodor!'))
```